CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

División de Tecnologías para la Integración Ciber-Humana

Departamento de Innovación Basada en la Información y el Conocimiento

Práctica 3: Red Neuronal Multicapa

Clasificación multiclase mediante retropropagación

Material complementario, fundamentos teóricos y descripción de la práctica

Asignatura	Sistemas Inteligentes 3 – Redes Neuronales Artificiales
Tema	Perceptrón multicapa, retropropagación y clasificación multiclase
Producto	Código fuente + reporte técnico
Base de trabajo	Misma base de altura y peso utilizada en la Práctica 2

Idea central de la práctica

Retomar el problema de altura, peso e índice de masa corporal de la práctica de ADALINE, pero conservar ahora las cuatro categorías originales: **bajo peso, peso normal, sobrepeso y obesidad**. El alumno deberá diseñar, programar y entrenar desde cero una red neuronal multicapa utilizando el algoritmo de retropropagación, y analizar cómo una capa oculta permite construir fronteras de decisión no lineales.

1. Propósito y resultados de aprendizaje

En la práctica anterior, ADALINE permitió estudiar una neurona lineal entrenada mediante una función de costo y descenso de gradiente. En esta práctica se extiende esa idea a una **red neuronal multicapa** y se introduce el algoritmo de **retropropagación del error** como procedimiento eficiente para calcular los gradientes de los pesos de todas las capas [1, 2, 3, 4, 5].

Al finalizar la práctica, el alumno deberá ser capaz de:

- diseñar una arquitectura multicapa apropiada para un problema de cuatro clases;
- implementar la propagación hacia adelante de una red neuronal;
- formular una función de error para múltiples neuronas de salida;
- calcular los gradientes locales de la capa de salida y de una capa oculta;
- implementar el algoritmo de retropropagación sin utilizar bibliotecas de redes neuronales;

- entrenar la red mediante gradiente estocástico, mini-lote o lote;
- visualizar las regiones de decisión generadas por la red;
- evaluar la generalización sobre patrones no utilizados durante el entrenamiento.

Regla fundamental de la práctica

El **IMC se utilizará únicamente para construir la etiqueta deseada**. Las entradas de la red serán exclusivamente **altura y peso**. Si el IMC se proporciona como entrada, la red recibe directamente la variable con la que se definieron las clases y se pierde el propósito de aprender las fronteras a partir de las variables originales.

2. Continuidad con la práctica de ADALINE

La entrada seguirá siendo

$$\mathbf{X}(k) = \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \begin{bmatrix} \text{altura}(k) \\ \text{peso}(k) \end{bmatrix}. \tag{1}$$

En ADALINE se utilizaron dos grupos y una sola frontera lineal. Ahora se conservarán las cuatro categorías:

$$\begin{aligned} &C_1 : \text{Bajo peso}, && \text{IMC} < 18.5, \\ &C_2 : \text{Peso normal}, && 18.5 \leq \text{IMC} < 25, \\ &C_3 : \text{Sobrepeso}, && 25 \leq \text{IMC} < 30, \\ &C_4 : \text{Obesidad}, && \text{IMC} \geq 30. \end{aligned} \tag{2}$$

Debido a que

$$\text{IMC} = \frac{m}{h^2}, \tag{3}$$

las fronteras ideales en el plano altura–peso están dadas por

$$\boxed{m = 18.5h^2}, \qquad \boxed{m = 25h^2}, \qquad \boxed{m = 30h^2}. \tag{4}$$

Por tanto, una única frontera lineal ya no es una representación adecuada del problema. La capa oculta permite combinar transformaciones no lineales y construir regiones de decisión más flexibles.

3. Fundamentos de la red neuronal multicapa

3.1. Arquitectura mínima

La red deberá contener, como mínimo:

$$\boxed{2 \text{ entradas} \longrightarrow N_h \text{ neuronas ocultas} \longrightarrow 4 \text{ neuronas de salida}}. \tag{5}$$

El número de neuronas ocultas N_h deberá ser seleccionado y justificado por el alumno. Se recomienda comenzar con una arquitectura pequeña y aumentar su tamaño solamente si los resultados lo requieren.

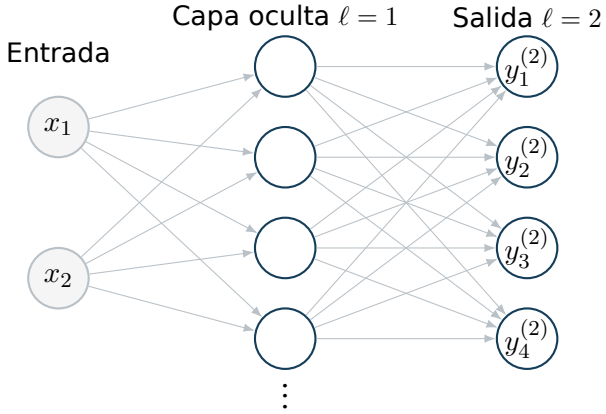


Figura 1: Arquitectura conceptual de una red $2 - N_h - 4$. El dibujo muestra algunas neuronas ocultas para ilustrar la notación; en $w_{ji}^{(\ell)}$, i identifica la neurona de origen y j la neurona destino de la capa ℓ.

3.2. Notación utilizada

Se utilizará la convención de los apuntes de clase:

$$\boxed{w_{ji}^{(\ell)}} \tag{6}$$

donde:

- ℓ es la capa a la que pertenece la neurona que recibe la conexión;
- j identifica la **neurona destino**;
- i identifica la **entrada a esa neurona**, es decir, la neurona de la capa anterior.

Así, $w_{ji}^{(1)}$ conecta la entrada i con la neurona oculta j, mientras que $w_{ji}^{(2)}$ conecta la neurona oculta i con la neurona de salida j.

La salida de la neurona j en la capa ℓ será $y_j^{(\ell)}(k)$, y su potencial local o entrada neta será $v_j^{(\ell)}(k)$. Para simplificar la propagación, se define

$$y_i^{(0)}(k) = x_i(k). \tag{7}$$

Interpretación tipo teoría de control / programación

El índice k identifica la presentación o actualización actual. La forma general de aprendizaje conserva la estructura

$$\text{parámetro nuevo} = \text{parámetro actual} + \text{corrección}.$$

Por ejemplo,

$$w_{ji}^{(\ell)}(k+1) = w_{ji}^{(\ell)}(k) + \Delta w_{ji}^{(\ell)}(k).$$

3.3. Propagación hacia adelante

Para una neurona j de la capa oculta:

$$v_j^{(1)}(k) = \sum_{i=1}^{2} w_{ji}^{(1)}(k) y_i^{(0)}(k) + b_j^{(1)}(k), \tag{8}$$

$$y_j^{(1)}(k) = \varphi^{(1)}\left(v_j^{(1)}(k)\right). \tag{9}$$

Para una neurona j de la capa de salida:

$$v_j^{(2)}(k) = \sum_{i=1}^{N_h} w_{ji}^{(2)}(k) y_i^{(1)}(k) + b_j^{(2)}(k), \tag{10}$$

$$y_j^{(2)}(k) = \varphi^{(2)}\left(v_j^{(2)}(k)\right), \qquad j = 1, 2, 3, 4. \tag{11}$$

La función de activación deberá ser diferenciable. Entre las opciones apropiadas para esta práctica se encuentran la sigmoide logística y la tangente hiperbólica:

$$\varphi(v) = \frac{1}{1 + e^{-av}}, \qquad \varphi'(v) = a\,\varphi(v)\,[1 - \varphi(v)], \tag{12}$$

$$\varphi(v) = \tanh(av), \qquad \varphi'(v) = a\,[1 - \varphi^2(v)]. \tag{13}$$

El alumno deberá indicar qué función utiliza en cada capa y programar explícitamente su derivada.

3.4. Representación de las cuatro clases

Con cuatro neuronas de salida, una opción natural es utilizar codificación *one-hot*. Si la activación de salida tiene rango $[0, 1]$:

$$\mathbf{d}_{C_1} = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}^T, \qquad \mathbf{d}_{C_2} = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}^T, \tag{14}$$

$$\mathbf{d}_{C_3} = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}^T, \qquad \mathbf{d}_{C_4} = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}^T. \tag{15}$$

Si se utiliza una activación bipolar, puede emplearse una codificación equivalente con $+1$ para la clase correcta y -1 para las demás. La codificación debe ser consistente con el rango de la función de activación seleccionada.

La clase predicha se obtendrá a partir de la neurona con mayor salida:

$$\boxed{\hat{C}(k) = \underset{j \in \{1,2,3,4\}}{\arg\max}\; y_j^{(2)}(k)}. \tag{16}$$

3.5. Función de error

Para cada neurona de salida j:

$$e_j(k) = d_j(k) - y_j^{(2)}(k). \tag{17}$$

La energía instantánea del error para el patrón k será

$$\boxed{E(k) = \frac{1}{2}\sum_{j=1}^{4} e_j^2(k)}. \tag{18}$$

Para una época completa de N patrones puede calcularse

$$\boxed{E_{\text{av}} = \frac{1}{N}\sum_{k=1}^{N} E(k)}. \tag{19}$$

El objetivo del entrenamiento es reducir E_{av} modificando los pesos y sesgos de todas las capas.

4. Algoritmo de retropropagación

4.1. Regla general de actualización

Para cualquier peso de la red:

$$w_{ji}^{(\ell)}(k+1) = w_{ji}^{(\ell)}(k) + \Delta w_{ji}^{(\ell)}(k), \tag{20}$$

con

$$\Delta w_{ji}^{(\ell)}(k) = -\eta \frac{\partial E(k)}{\partial w_{ji}^{(\ell)}(k)}. \tag{21}$$

El algoritmo de retropropagación utiliza la regla de la cadena para calcular estas derivadas desde la salida hacia las capas anteriores [1, 2].

4.2. Gradiente local de una neurona de salida

Para una neurona j de la capa de salida, se define el gradiente local como

$$\delta_j^{(2)}(k) = -\frac{\partial E(k)}{\partial v_j^{(2)}(k)}. \tag{22}$$

Aplicando la regla de la cadena:

$$\delta_j^{(2)}(k) = -\frac{\partial E(k)}{\partial e_j(k)} \frac{\partial e_j(k)}{\partial y_j^{(2)}(k)} \frac{\partial y_j^{(2)}(k)}{\partial v_j^{(2)}(k)} \tag{23}$$

$$= e_j(k)\, \varphi'^{(2)}\left(v_j^{(2)}(k)\right). \tag{24}$$

Por tanto,

$$\boxed{\delta_j^{(2)}(k) = e_j(k)\, \varphi'^{(2)}\left(v_j^{(2)}(k)\right)}. \tag{25}$$

Como

$$\frac{\partial v_j^{(2)}(k)}{\partial w_{ji}^{(2)}(k)} = y_i^{(1)}(k), \tag{26}$$

la actualización del peso queda

$$\boxed{w_{ji}^{(2)}(k+1) = w_{ji}^{(2)}(k) + \eta\, \delta_j^{(2)}(k) y_i^{(1)}(k)}. \tag{27}$$

Para el sesgo:

$$\boxed{b_j^{(2)}(k+1) = b_j^{(2)}(k) + \eta\, \delta_j^{(2)}(k)}. \tag{28}$$

4.3. Gradiente local de una neurona oculta

En una neurona oculta no existe una salida deseada conocida directamente. Su contribución al error se determina a través de las neuronas de la siguiente capa. Para una red con una sola capa oculta:

$$\boxed{\delta_j^{(1)}(k) = \varphi'^{(1)}\left(v_j^{(1)}(k)\right) \sum_{r=1}^{4} \delta_r^{(2)}(k) w_{rj}^{(2)}(k)}. \tag{29}$$

La suma ponderada

$$\sum_{r=1}^{4} \delta_r^{(2)}(k) w_{rj}^{(2)}(k) \tag{30}$$

representa la forma en que el error de las neuronas de salida se propaga hacia atrás hasta la neurona oculta j.

La actualización de los pesos de la capa oculta será

$$\boxed{w_{ji}^{(1)}(k+1) = w_{ji}^{(1)}(k) + \eta\, \delta_j^{(1)}(k) y_i^{(0)}(k)}. \tag{31}$$

Como $y_i^{(0)}(k) = x_i(k)$:

$$w_{ji}^{(1)}(k+1) = w_{ji}^{(1)}(k) + \eta\, \delta_j^{(1)}(k) x_i(k). \tag{32}$$

Para el sesgo:

$$\boxed{b_j^{(1)}(k+1) = b_j^{(1)}(k) + \eta\, \delta_j^{(1)}(k)}. \tag{33}$$

Secuencia conceptual de Backpropagation

Forward pass $\rightarrow$ error de salida $\rightarrow \delta^{(2)} \rightarrow \delta^{(1)} \rightarrow$ actualización de pesos y sesgos.

La información fluye hacia adelante para calcular la salida, mientras que el gradiente del error se propaga hacia atrás para calcular las correcciones.

4.4. Entrenamiento estocástico, mini-lote o lote

El alumno podrá utilizar cualquiera de las tres modalidades estudiadas. **No es obligatorio implementar las tres**, pero deberá indicar claramente cuál eligió y programarla desde cero.

Entrenamiento estocástico (SGD)

Los pesos se actualizan después de cada patrón mediante las expresiones (27) y (31). Es recomendable mezclar los patrones al comienzo de cada época.

Entrenamiento por mini-lotes

Para un mini-lote $\mathcal{B}$ de tamaño M, las correcciones pueden acumularse y promediarse:

$$w_{ji}^{(\ell)} \leftarrow w_{ji}^{(\ell)} + \eta \frac{1}{M} \sum_{k \in \mathcal{B}} \delta_j^{(\ell)}(k) y_i^{(\ell-1)}(k). \tag{34}$$

Entrenamiento por lote (Batch)

Se acumulan los gradientes de los N patrones antes de modificar los parámetros:

$$w_{ji}^{(\ell)} \leftarrow w_{ji}^{(\ell)} + \eta \frac{1}{N} \sum_{k=1}^{N} \delta_j^{(\ell)}(k) y_i^{(\ell-1)}(k). \tag{35}$$

Cuidado al implementar mini-lote o batch

Los δ de un patrón deben calcularse con los pesos correspondientes al estado actual de la red. Si se acumulan gradientes, los pesos no deben modificarse hasta terminar el mini-lote o el lote completo.

5. Descripción de la práctica

5.1. Base de datos

La práctica utilizará como punto de partida la **misma base de datos de altura y peso empleada en la Práctica 2**. El alumno deberá conservar la referencia y documentación de su fuente. Si modifica la base, elimina registros o realiza un filtrado adicional, deberá justificarlo.

Siempre que sea posible, se recomienda conservar también la misma división entrenamiento–prueba utilizada en la práctica anterior para facilitar la comparación entre ADALINE y la red multicapa.

5.2. Construcción de las cuatro etiquetas

El IMC deberá calcularse con las unidades físicas originales:

$$\mathrm{IMC}(k) = \frac{\mathrm{peso}(k)\ [\mathrm{kg}]}{(\mathrm{altura}(k)\ [\mathrm{m}])^2}. \tag{36}$$

Después se asignará una de las cuatro clases de (2). El IMC no se incluirá en el vector de entrada.

5.3. Escalamiento de las entradas

Deberá reutilizarse alguno de los métodos estudiados en la práctica anterior. Como mínimo, el alumno debe comprender y poder implementar:

$$\text{Min–Max:} \qquad \bar{x}_j(k) = \frac{x_j(k) - x_{j,\text{mín}}}{x_{j,\text{máx}} - x_{j,\text{mín}}}, \tag{37}$$

$$\text{Standard Scaling:} \qquad z_j(k) = \frac{x_j(k) - \mu_j}{\sigma_j}. \tag{38}$$

Los parámetros de escalamiento deben calcularse **únicamente con el conjunto de entrenamiento** y reutilizarse sin cambios para prueba y generalización. El alumno deberá reportar el método utilizado y sus parámetros.

5.4. Diseño de la red

El alumno deberá proponer una arquitectura de al menos una capa oculta y cuatro neuronas de salida. El reporte deberá especificar:

- número de neuronas ocultas N_h;
- función de activación de la capa oculta;
- función de activación de la capa de salida;
- codificación de las cuatro clases;
- método de inicialización de pesos y sesgos;
- razón de aprendizaje η;
- modalidad de gradiente: estocástico, mini-lote o lote;
- número máximo de épocas y criterio de paro.

Recomendación para iniciar

Una arquitectura pequeña, por ejemplo $2-4-4$ o $2-6-4$, es un punto de partida razonable para experimentar. No debe asumirse que aumentar el número de neuronas ocultas mejora automáticamente la generalización.

5.5. Implementación obligatoria desde cero

No se permite utilizar una biblioteca que implemente directamente la red o su entrenamiento.

Permitido	**No permitido**
Arreglos, álgebra básica, lectura de archivos y generación de gráficas.	TensorFlow, Keras, PyTorch, `sklearn.neural_network` u otra implementación de MLP.
Programar manualmente funciones de activación y sus derivadas.	Utilizar funciones de entrenamiento automático o diferenciación automática para obtener los gradientes.
Implementar manualmente forward pass, error, δ, backpropagation y actualización.	Sustituir el algoritmo por un clasificador ya implementado.
Usar bibliotecas para visualizar datos y resultados.	Utilizar el IMC como entrada de la red.

5.6. Entrenamiento

El alumno deberá registrar, al menos una vez por época, la energía promedio (19). La curva de aprendizaje deberá mostrar

$$\boxed{E_{\text{av}}(q) \quad \text{vs.} \quad \text{época } q}. \tag{39}$$

Además, deberá vigilar que el error no se vuelva indefinido o crezca sin límite. Si ocurre, deberá analizar posibles causas: razón de aprendizaje excesiva, inicialización inadecuada, error en las derivadas, falta de escalamiento o errores en la implementación de backpropagation.

5.7. Regiones y fronteras de decisión

Una de las gráficas principales de la práctica será el mapa de decisión en el plano altura–peso. Para construirlo:

1. generar una malla de puntos (h, m) que cubra el intervalo de la base de datos;
2. aplicar a cada punto el mismo escalamiento utilizado durante el entrenamiento;
3. propagar cada punto por la red;
4. asignar la clase mediante (16);
5. graficar las cuatro regiones obtenidas y superponer los patrones reales.

Sobre la misma gráfica deberán incluirse, como referencia, las fronteras teóricas

$$m = 18.5h^2, \qquad m = 25h^2, \qquad m = 30h^2. \tag{40}$$

El objetivo no es que la red reproduzca exactamente estas curvas, sino analizar qué representación aprendió a partir de los datos disponibles.

5.8. Pruebas de generalización

Los datos de prueba no deberán utilizarse para ajustar pesos, seleccionar gradientes ni recalcular parámetros de escalamiento. Sobre este conjunto se deberá reportar:

- número total de patrones;
- número de clasificaciones correctas e incorrectas;
- exactitud global;
- matriz de confusión 4×4;
- exactitud o tasa de acierto por clase.

La exactitud global se calcula como

$$\text{Accuracy} = \frac{\text{clasificaciones correctas}}{\text{número total de patrones de prueba}} \times 100\,\%. \tag{41}$$

La matriz de confusión tendrá la forma

	$\widehat{C}_1$	$\widehat{C}_2$	$\widehat{C}_3$	$\widehat{C}_4$
C_1				
C_2				
C_3				
C_4				

5.9. Análisis de errores

El alumno deberá identificar patrones mal clasificados y revisar su ubicación respecto a los umbrales de IMC. Es razonable esperar que parte de los errores se concentren cerca de las fronteras entre categorías. También deberá comprobar si existe desbalance entre las cuatro clases y discutir cómo afecta los resultados.

6. Algoritmo mínimo de referencia

El siguiente esquema no es código y no debe copiarse como una implementación completa; resume el flujo que debe programarse:

1. Cargar altura y peso.
2. Calcular IMC con las unidades originales y construir las cuatro etiquetas.
3. Separar entrenamiento y prueba.
4. Calcular los parámetros de escalamiento usando solamente entrenamiento.
5. Escalar entrenamiento y prueba con los mismos parámetros.
6. Elegir N_h, funciones de activación, η, modalidad de gradiente y criterio de paro.
7. Inicializar $w_{ji}^{(1)}$, $w_{ji}^{(2)}$, $b_j^{(1)}$ y $b_j^{(2)}$.
8. Para cada época:
   a) realizar propagación hacia adelante;
   b) calcular $e_j(k)$ y $E(k)$;

c) calcular $\delta_j^{(2)}(k)$;

d) calcular $\delta_j^{(1)}(k)$;

e) acumular o aplicar las correcciones según SGD, mini-lote o batch;

f) calcular E_{av} de la época.

9. Evaluar el conjunto de prueba sin modificar los parámetros.
10. Generar la curva de error, la matriz de confusión y el mapa de regiones de decisión.

7. Resultados mínimos a entregar

Formato del reporte

El reporte de la práctica deberá presentarse utilizando un formato oficial de **IEEE**, ya sea **IEEE Journal** o **IEEE Conference**, a elección del alumno. Se deberá respetar la estructura, tipografía, márgenes, organización en columnas y estilo de referencias bibliográficas de la plantilla seleccionada. El documento final deberá entregarse en formato PDF.

#	Evidencia requerida
1	Referencia de la base de datos utilizada y resumen de su preparación.
2	Definición de las cuatro clases y distribución de patrones por clase.
3	División entrenamiento–prueba y método de escalamiento utilizado.
4	Diagrama y descripción de la arquitectura de la red.
5	Ecuaciones o pseudocódigo de la propagación hacia adelante y retropropagación implementadas.
6	Valores de η, número de neuronas ocultas, funciones de activación, modalidad de gradiente, inicialización y criterio de paro.
7	Gráfica de E_{av} contra época.
8	Mapa de regiones de decisión en altura–peso, con los patrones y las tres fronteras teóricas de IMC.
9	Resultados sobre entrenamiento y prueba.
10	Matriz de confusión 4×4 y exactitud por clase.
11	Ejemplos de patrones mal clasificados y análisis de su posición respecto a las fronteras.
12	Código fuente completo, ejecutable y comentado.
13	Conclusiones técnicas: qué permitió resolver la capa oculta que ADALINE no podía representar adecuadamente.

8. Preguntas de análisis para el reporte

1. ¿Por qué una sola neurona lineal no es suficiente para representar adecuadamente las cuatro categorías en el plano altura–peso?

2. ¿Qué papel desempeña la función de activación no lineal de la capa oculta?
3. Explique con sus propias palabras qué representa $\delta_j^{(2)}(k)$ y qué representa $\delta_j^{(1)}(k)$.
4. ¿Por qué el gradiente de una neurona oculta depende de los pesos que la conectan con la capa siguiente?
5. ¿Qué diferencias observa entre el error de entrenamiento y el desempeño sobre el conjunto de prueba?
6. ¿En qué zonas del plano altura–peso se concentran los errores de clasificación? ¿Coinciden con las fronteras de IMC?
7. ¿Qué efecto tuvo el número de neuronas ocultas sobre el entrenamiento y la generalización?
8. ¿Qué problemas aparecen si η es demasiado pequeño o demasiado grande?
9. ¿Por qué los parámetros de normalización deben calcularse únicamente con los datos de entrenamiento?
10. Compare conceptualmente esta práctica con ADALINE: ¿qué cambió en el modelo y qué cambió en el algoritmo de aprendizaje?

9. Experimento complementario recomendado

Como extensión, se recomienda entrenar al menos dos arquitecturas con diferente número de neuronas ocultas, por ejemplo

$$2-4-4 \qquad \text{y} \qquad 2-8-4, \tag{42}$$

utilizando la misma división de datos y el mismo método de escalamiento. La comparación debe considerar no solamente el error final de entrenamiento, sino también la exactitud de prueba y la forma de las regiones de decisión.

Idea que debe quedar al finalizar

La red multicapa no reemplaza la idea de descenso de gradiente estudiada con ADALINE; la generaliza. La dificultad adicional consiste en que los pesos de las capas ocultas afectan al error de manera indirecta. La regla de la cadena permite atribuir a cada neurona una contribución local al gradiente y hace posible entrenar toda la red mediante retropropagación.

10. Referencias

Referencias

[1] *Algoritmo de retropropagación*. Apuntes de clase proporcionados en el curso de Redes Neuronales Artificiales, 2026. En particular: definición del error cuadrático, regla de la cadena, gradiente local de la capa de salida y propagación del gradiente hacia capas ocultas.

[2] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Pearson, 2009. Capítulo 4: Multilayer Perceptrons, especialmente propagación hacia adelante, algoritmo de back-propagation, aprendizaje en línea y por lote.

[3] B. Mehlig, *Machine Learning with Neural Networks: An Introduction for Scientists and Engineers*, Cambridge University Press, 2021. Capítulos 5 y 6: perceptrones multicapa, regla de la cadena, retropropagación y descenso de gradiente estocástico.

[4] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, Springer, 2018. Capítulos 1 y 3: entrenamiento mediante backpropagation, grafos computacionales, mini-batch y aspectos prácticos de optimización.

[5] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. Capítulos 6 y 8: redes feedforward profundas, algoritmos de diferenciación/back-propagation y optimización basada en gradiente.